МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине

«Операционные системы»

«Файловая система»

Работу выполнил:
Студенты группы P3411
Болорболд Аригуун

Лектор:
Клименков Сергей Викторович
Практик:
Смирнов Виктор Игоревич

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
   1.1. Часть 1. Знакомство с простым модулем 3
   1.2. Часть 2. Подготовка файловой системы 4
   1.3. Часть 3. Вывод файлов и директорий 7
   1.4. Часть 4. Навигация по директориям 10
   1.5. Часть 5. Создание и удаление файлов 10
   1.6. Часть 6. Реализация с хранилищем данных в RAM 12
   1.7. Часть 7. Создание и удаление директорий 12
   1.8. Часть 8*. Чтение и запись в файлы 12
   1.9. Часть 9*. Жёсткие ссылки 13
   1.10. Часть 10*. Сервер файловой системы 13
2. Ход работы 15
   2.1. Часть 1. Знакомство с простым модулем 15
   2.2. Часть 2. Подготовка файловой системы 15
   2.3. Часть 3. Вывод файлов и директорий 17
   2.4. Часть 4. Навигация по директориям 19
   2.5. Часть 5. Создание и удаление файлов 19
   2.6. Часть 6. Реализация с хранилищем данных в RAM 21
   2.7. Часть 7. Создание и удаление директорий 21
   2.8. Часть 8*. Чтение и запись в файлы 22
   2.9. Часть 9*. Жёсткие ссылки 23
   2.10. Часть 10*. Сервер файловой системы 24
3. Вывод 25

1. Текст задания

Ядро ОС `Linux` — монолитное. Это означает, что все его части работают в общем адресном пространстве. Однако, это не означает, что для добавления какой-то возможности необходимо полностью перекомпилировать ядро. Новую функциональность можно добавить в виде модуля ядра. Такие модули можно легко загружать и выгружать по необходимости прямо во время работы системы. Подробнее смотри глава 2.10 в книге «Операционные системы: внутренняя структура и принципы проектирования», Вильям Столлингс.

С помощью модулей можно реализовать свои файловые системы, причём со стороны пользователя такая файловая система ничем не будет отличаться от ext4 или NTFS. В этом задании мы с Вами реализуем простую файловую систему: все файлы будут храниться в оперативной памяти компьютера или на удаленном сервере (по желанию), однако пользователь сможет пользоваться ими точно так же, как и файлами на жёстком диске.

Мы рекомендуем при выполнении этого задания использовать отдельную виртуальную машину: любая ошибка может вывести всю систему из строя, и вы можете потерять ваши данные. Именно виртуальную машину, а не контейнер типа Docker (почему?).

1.1. Часть 1. Знакомство с простым модулем

Давайте научимся компилировать и подключать тривиальный модуль. Для компиляции модулей ядра нам понадобятся утилиты для сборки и заголовочные файлы для конкретной версии ядра. Установить их можно так:

```
sudo apt-get install build-essential linux-headers-`uname -r`
```

Мы уже подготовили основу для вашего будущего модуля в файлах vtfs.c и Makefile. Познакомьтесь с ней.

Ядру для работы с модулем достаточно двух функций — одна должна инициализировать модуль, а вторая — очищать результаты его работы. Они указываются с помощью `module_init` и `module_exit`.

Важное отличие кода для ядра `Linux` от user-space-кода — в отсутствии в нём стандартной библиотеки `libc`. Например, в ней же находится функция `printf`. Чтобы получить обратную связь от программы, мы можем печатать данные в системный лог с помощью функции printk.

В Makefile указано, что наш модуль `vtfs` состоит из одной единицы трансляции — `vtfs`. Вы можете самостоятельно добавлять новые единицы,

чтобы декомпозировать ваш код удобным образом.

Соберём модуль.

```
$ make
```

Сборка модуля отличается от сборки обычных программ тем, что при этом происходит некоторая «магия». А именно, Makefile обрабатывается не обычным make, а особым, с дополнительными целями и переменными, так же выполняются другие незаметные операции.

Если наш код скомпилировался успешно, в директории `source` появится файл `vtfs.ko` — это и есть наш модуль. Осталось загрузить его в ядро. Загружается именно файл, поэтому указывается путь до содержимого модуля (с расширением `.ko`).

```
$ sudo insmod source/vtfs.ko
```

Однако, мы не увидели нашего сообщения. Оно печатается не в терминал, а в системный лог — его можно увидеть командой `dmesg`.

```
$ dmesg
<...>
[ 123.456789] [vtfs] VTFS joined the kernel
```

Для выгрузки модуля нам понадобится команда `rmmod`. Выгрузка уже происходит по имени модуля (без расширения `.ko`, см. команду `lsmod`).

```
$ sudo rmmod vtfs
$ dmesg
<...>
[ 123.987654] [vtfs] VTFS left the kernel
```

1.2. Часть 2. Подготовка файловой системы

Мы собираемся сделать простую файловую систему, которая сначала будет хранить данные в оперативной памяти, в самом начале даже просто симулировать наличие файлов, а позже будет предложено реализовать отправку по сети и хранение их на удаленной машине.
Операционная система предоставляет две функции для управления файловыми системами:

- `register_filesystem` — сообщает о появлении нового драйвера файловой системы.
- `unregister_filesystem` — удаляет драйвер файловой системы.

В этой части мы начнём работать с несколькими структурами ядра:

- `inode` — описание метаданных файла: имя файла, расположение, тип файла (в нашем случае — регулярный файл или директория)

- `dentry` — описание директории: список `inode` внутри неё, информация о родительской директории, …
- `super_block` — описание всей файловой системы: информация о корневой директории, …

Функции `register_filesystem` и `unregister_filesystem` принимают структуру с описанием файловой системы. Начнём с такой:

```
struct file_system_type vtfs_fs_type = {
  .name = "vtfs",
  .mount = vtfs_mount,
  .kill_sb = vtfs_kill_sb,
};
```

Для монтирования файловой системы в этой структуре мы добавили два поля. Первое — `mount` — указатель на функцию, которая вызывается при монтировании.
Например, она может выглядеть так:

```
struct dentry* vtfs_mount(
  struct file_system_type* fs_type,
  int flags,
  const char* token,
  void* data
) {
  struct dentry* ret = mount_nodev(fs_type, flags, data,
vtfs_fill_super);
  if (ret == NULL) {
    printk(KERN_ERR "Can't mount file system");
  } else {
    printk(KERN_INFO "Mounted successfuly");
  }
  return ret;
}
```

Эта функция будет вызываться всякий раз, когда пользователь будет монтировать нашу файловую систему. Например, он может это сделать командой `mount`:

```
sudo mount -t vtfs "<token>" "<path>"
```

Опция `-t` нужна для указания имени файловой системы — именно оно указывается в поле name. Также мы передаём токен, полученный в прошлой части, и локальную директорию, в которую ФС будет примонтирована. Обратите внимание, что эта директория должна быть пуста.

Мы используем функцию `mount_nodev`, поскольку наша файловая система не хранится на каком-либо физическом устройстве:

```
struct dentry* mount_nodev(
  struct file_system_type* fs_type,
  int flags,
  void* data,
  int (*fill_super)(struct super_block*, void*, int)
);
```

Последний её аргумент — указатель на функцию `fill_super`. Эта функция должна заполнять структуру `super_block` информацией о файловой системе. Давайте начнём с такой функции:

```
int vtfs_fill_super(struct super_block *sb, void *data, int silent)
{
  struct inode* inode = vtfs_get_inode(sb, NULL, S_IFDIR, 1000);

  sb->s_root = d_make_root(inode);
  if (sb->s_root == NULL) {
    return -ENOMEM;
  }

  printk(KERN_INFO "return 0\n");
  return 0;
}
```

Аргументы `data` и `silent` нам не понадобятся. В этой функции мы используем ещё одну (пока) неизвестную функцию — `vtfs_get_inode`. Она будет создавать новую структуру `inode`, в нашем случае — для корня файловой системы:

```
struct inode* vtfs_get_inode(
  struct super_block* sb,
  const struct inode* dir,
  umode_t mode,
  int i_ino
) {
  struct inode *inode = new_inode(sb);
  if (inode != NULL) {
    inode_init_owner(inode, dir, mode);
  }

  inode->i_ino = i_ino;
  return inode;
}
```

Давайте поймём, что эта функция делает. Файловой системе нужно знать, где находится корень файловой системы. Для этого в поле `s_root` мы записываем результат функции `d_make_root`, передавая ему корневую `inode`. Для определенности корневая директория всегда будет иметь номер `1000`.

Для создания новой `inode` используем функцию `new_inode`. Кроме этого, с помощью функции `inode_init_owner` зададим тип ноды — укажем, что это директория.

На самом деле, `umode_t` содержит битовую маску, все значения которой доступны в заголовочном файле `linux/stat.h` — она задаёт тип объекта и права доступа.

Второе поле, которое мы определили в `file_system_type` — поле `kill_sb` — указатель на функцию, которая вызывается при отмонтировании файловой системы. В нашем случае ничего делать не нужно:

```
void vtfs_kill_sb(struct super_block* sb) {
  printk(KERN_INFO "vtfs super block is destroyed. Unmount
successfully.\n");
}
```

Не забудьте зарегистрировать файловую систему в функции инициализации модуля, и удалять её при очистке модуля. Наконец, соберём и примонтируем нашу файловую систему:

```
sudo make
sudo insmod vtfs.ko
sudo mount -t vtfs "TODO" /mnt/vt
```

Если вы всё правильно сделали, ошибок возникнуть не должно. Тем не менее, перейти в директорию `/mnt/vt` не выйдет — ведь мы ещё не реализовали никаких функций для навигации по ФС.

Теперь отмонтируем файловую систему:

```
$ sudo umount /mnt/vt
```

1.3. Часть 3. Вывод файлов и директорий

В прошлой части мы закончили на том, что не смогли перейти в директорию:

```
$ sudo mount -t vtfs "TODO" /mnt/vt
$ cd /mnt/vt
-bash: cd: /mnt/vt: Not a directory
```

Чтобы это исправить, необходимо реализовать некоторые методы для работы с `inode`.

Чтобы эти методы вызывались, в поле `i_op` нужной нам ноды необходимо записать структуру `inode_operations`. Например, такую:

```
struct inode_operations vtfs_inode_ops = {
  .lookup = vtfs_lookup,
};
```

Первая функция, которую мы реализуем — `lookup`. Именно она позволяет операционной системе определять, что за сущность описывается данной нодой.
Сигнатура функции должна быть такой:

```
struct dentry* vtfs_lookup(
  struct inode* parent_inode,  // родительская нода
  struct dentry* child_dentry, // объект, к которому мы пытаемся
получить доступ
  unsigned int flag            // неиспользуемое значение
);
```

Пока ничего не будем делать: просто вернём `NULL`. Если мы заново попробуем повторить переход в директорию, у нас ничего не получится — но уже по другой причине:

```
$ cd /mnt/vt
-bash: cd: /mnt/vt: Permission denied
```

Решите эту проблему. Пока сложной системы прав у нас не будет — у всех объектов в файловой системе могут быть права `777`. В итоге должно получиться что-то такое:

```
$ ls -l /mnt/
total 0
drwxrwxrwx 1 root root 0 Oct 24 15:52 vt
```

После этого мы сможем перейти в `/mnt/vt`, но не можем вывести содержимое директории. На этот раз нам понадобится не `i_op`, а `i_fop` — структура типа `file_operations`. Реализуем в ней первую функцию — `iterate`.

```
struct file_operations vtfs_dir_ops = {
  .iterate = vtfs_iterate,
};
```

Эта функция вызывается только для директорий и выводит список объектов в ней (нерекурсивно): для каждого объекта вызывается функция `dir_emit`, в которую передаётся имя объекта, номер ноды и его тип.

Пример функции `vtfs_iterate` приведён ниже:

```
int vtfs_iterate(struct file* filp, struct dir_context* ctx) {
  char fsname[10];
  struct dentry* dentry = filp->f_path.dentry;
  struct inode* inode   = dentry->d_inode;
  unsigned long offset  = filp->f_pos;
  int stored            = 0;
  ino_t ino             = inode->i_ino;

  unsigned char ftype;
  ino_t dino;
  while (true) {
    if (ino == 100) {
      if (offset == 0) {
        strcpy(fsname, ".");
        ftype = DT_DIR;
        dino = ino;
      } else if (offset == 1) {
        strcpy(fsname, "..");
        ftype = DT_DIR;
        dino = dentry->d_parent->d_inode->i_ino;
      } else if (offset == 2) {
        strcpy(fsname, "test.txt");
        ftype = DT_REG;
        dino = 101;
      } else {
        return stored;
      }
    }
  }
}
```

Попробуем снова получить список файлов:

```
$ ls /mnt/vt
ls: cannot access '/mnt/vt/test.txt': No such file or directory
test.txt
```

Эта ошибка возникла из-за того, что `lookup` работает только для корневой директории — но не для файла `test.txt`. Это мы исправим в следующих частях.

1.4. Часть 4. Навигация по директориям

Теперь мы хотим научиться переходить по директориям. На этом шаге функцию `vtfs_lookup` придётся немного расширить: если такой файл есть, нужно вызывать функцию `d_add`, передавая ноду файла. Например, так:

```
struct dentry* vtfs_lookup(
  struct inode* parent_inode,
  struct dentry* child_dentry,
  unsigned int flag
) {
  ino_t root = parent_inode->i_ino;
  const char *name = child_dentry->d_name.name;
  if (root == 100 && !strcmp(name, "test.txt")) {
    struct inode *inode = vtfs_get_inode(parent_inode->i_sb, NULL,
S_IFREG, 101);
    d_add(child_dentry, inode);
  } else if (root == 100 && !strcmp(name, "dir")) {
    struct inode *inode = vtfs_get_inode(parent_inode->i_sb, NULL,
S_IFDIR, 200);
    d_add(child_dentry, inode);
  }
  return NULL;
}
```

1.5. Часть 5. Создание и удаление файлов

Теперь научимся создавать и удалять файлы. Добавим ещё два поля в `inode_operations` — `create` и `unlink`: Функция `vtfs_create` вызывается при создании файла и должна возвращать новую `inode` с помощью `d_add`, если создать файл получилось. Рассмотрим простой пример:

```
int vtfs_create(
  struct inode *parent_inode,
  struct dentry *child_dentry,
  umode_t mode,
  bool b
) {
  ino_t root = parent_inode->i_ino;
  const char *name = child_dentry->d_name.name;
  if (root == 100 && !strcmp(name, "test.txt")) {
    struct inode *inode = vtfs_get_inode(
        parent_inode->i_sb, NULL, S_IFREG | S_IRWXUGO, 101);
    inode->i_op = &vtfs_inode_ops;
    inode->i_fop = NULL;

    d_add(child_dentry, inode);
```

```
    mask |= 1;
  } else if (root == 100 && !strcmp(name, "new_file.txt")) {
    struct inode *inode = vtfs_get_inode(
        parent_inode->i_sb, NULL, S_IFREG | S_IRWXUGO, 102);
    inode->i_op = &vtfs_inode_ops;
    inode->i_fop = NULL;

    d_add(child_dentry, inode);
    mask |= 2;
  }
  return 0;
}
```

Чтобы проверить, как создаются файлы, воспользуемся утилитой `touch`:

```
$ touch test.txt
$ ls
test.txt
$ touch new_file.txt
$ ls
test.txt new_file.txt
```

Для удаления файлов определим ещё одну функцию — `vtfs_unlink`.

```
int vtfs_unlink(struct inode *parent_inode, struct dentry
*child_dentry) {
  const char *name = child_dentry->d_name.name;
  ino_t root = parent_inode->i_ino;
  if (root == 100 && !strcmp(name, "test.txt")) {
    mask &= ~1;
  } else if (root == 100 && !strcmp(name, "new_file.txt")) {
    mask &= ~2;
  }
  return 0;
}
```

Теперь у нас получится выполнять и команду `rm`.

```
$ ls
test.txt new_file.txt
$ rm test.txt
$ ls
new_file.txt
$ rm new_file.txt
$ ls
```

Обратите внимание, что утилита `touch` проверяет существование файла: для этого вызывается функция `lookup`.

1.6. Часть 6. Реализация с хранилищем данных в RAM

До сих пор мы писали лишь заглушку файловой системы — содержимое ФС было зашито в коде.

Теперь пришло время реализовать функции из предыдущих этапов, но используя оперативную память в качестве хранилища данных для нашей ФС. Например, можно хранить данные просто в массивах и связных списках.

Рекомендуем вам заранее выделить интерфейс файловой системы, чтобы упростить смену хранилища данных — это понадобится нам в следующих этапах.

1.7. Часть 7. Создание и удаление директорий

Следующая часть нашего задания — создание и удаление директорий. Добавим в `inode_operations` ещё два поля — `mkdir` и `rmdir`. Их сигнатуры можно найти тут.

1.8. Часть 8*. Чтение и запись в файлы

Реализуйте чтение из файлов и запись в файлы. Для этого вам понадобится структура `file_operations` не только для директорий, но и для обычных файлов. В неё вам понадобится добавить два поля — `read` и `write`.

Соответствующие функции имеют следующие сигнатуры:

```
ssize_t vtfs_read(
  struct file *filp, // файловый дескриптор
  char *buffer,      // буфер в user-space для чтения и записи
соответственно
  size_t len,        // длина данных для записи
  loff_t *offset     // смещение
);

ssize_t vtfs_write(
  struct file *filp,
  const char *buffer,
  size_t len,
  loff_t *offset
);
```

Обратите внимание, что просто так обратиться в `buffer` нельзя, поскольку он находится в `user-space`. Используйте специальные функции

для чтения и записи.

В результате вы сможете сделать вот так:

```
$ cat file1
hello world from file1
$ cat file2
file2 content here
$ echo "test" > file1
$ cat file1
test
```

Обратите внимание, что файл должен уметь содержать любые ASCII-символы с кодами от 0 до 127 включительно.

1.9. Часть 9*. Жёсткие ссылки

Вам необходимо поддержать возможность сослаться из разных мест файловой системы на одну и ту же `inode`.

Обратите внимание: сервер поддерживает жёсткие ссылки только для регулярных файлов, но не для директорий.

Для этого добавьте поле `link` в структуру `inode_operations`. Сигнатура соответствующей функции выглядит так:

```
int vtfs_link(
  struct dentry *old_dentry,
  struct inode *parent_dir,
  struct dentry *new_dentry
);
```

После реализации функции вы сможете выполнить следующие команды:

```
$ ln file1 file3
$ cat file1
hello world from file1
$ cat file3
hello world from file1
$ echo "test" > file1
$ rm file1
$ cat file3
test
```

1.10. Часть 10*. Сервер файловой системы

При текущей реализации содержимое файловой системы не переживает перезапуск компьютера. Давайте исправим это и будем хранить данные на каком-то долговечном носителе. Чтобы было интереснее, вам придется реализовать удаленную файловую систему (подобную `netfs`).

Вам нужно на любом языке программирования реализовать сервер вашей файловой системы. В качестве хранилища на сервере можете использовать СУБД. Например, это может быть связка Spring и PostgreSQL или ZIO и YDB.

Поскольку в ядре используются не совсем привычные функции для работы с сетью, вам предоставляется собственный HTTP-клиент в виде функции `vtfs_http_call` в файле [http.c](./source/http.c):

```
int64_t vtfs_http_call(
    const char *token,     // ваш токен
    const char *method,    // название метода без неймспейса fs
(list, create, ...)
    char *response_buffer, // буфер для сохранения ответа от
сервера
    size_t buffer_size,    // размер буфера
    size_t arg_size,       // количество аргументов
    // далее должны следовать 2 * arg_size аргументов типа const
char*
    // - пары param1, value1, param2, value2, ... — параметры
запроса
    ...
);
```

Функция возвращает 0, если запрос завершён успешно; положительное число — код ошибки из документации API, если сервер вернул ошибку; отрицательное число — код ошибки из [http.h](./source/http.h) или `errno-base.h` (`ENOMEM`, `ENOSPC`) в случае ошибки при выполнении запроса (отсутствие подключения, сбой в сети, некорректный ответ сервера, …).

2. Ход работы

2.1. Часть 1. Знакомство с простым модулем

Для того, чтобы нам получилось запустить модули ядра `Linux` без особых затруднении (вызванных в основном из-за `Windows`, к сожалению `WSL2` не помогает здесь), по совету задании будем запускать экземпляр ОС `Ubuntu` на `Qemu`. Сначала создаем образ диска для установки:

```
$ qemu-img create -f qcow2 ubuntu_disk.qcow2 20G
Formatting 'ubuntu_disk.qcow2', fmt=qcow2 cluster_size=65536
extended_l2=off compression_type=zlib size=21474836480
lazy_refcounts=off refcount_bits=16
```

Потом на его основе запускаем сам ОС:

```
qemu-system-x86_64 \
  -enable-kvm \
  -m 2G \
  -smp 2 \
  -hda ubuntu_disk.qcow2 \
  -cdrom ubuntu-24.04.3-desktop-amd64.iso \
  -device virtio-net-pci,netdev=net0 \ # Важно! Потом увидите
  -netdev user,id=net0 \
  -rtc base=localtime
```

2.2. Часть 2. Подготовка файловой системы

Сначала надо проинициировать файловую систему при запуске. Как было написано в задании, это делается через вызов функции `register_filesystem()` и `unregister_filesystem()`:

```
struct file_system_type vtfs_fs_type = {
    .name = "vtfs",
    .mount = vtfs_mount,
    .kill_sb = vtfs_kill_sb,
};

// ...

static int __init vtfs_init(void) {
  register_filesystem(&vtfs_fs_type);
  LOG("VTFS joined the kernel\n");
  return 0;
}

static void __exit vtfs_exit(void) {
  unregister_filesystem(&vtfs_fs_type);
```

```
  LOG("VTFS left the kernel\n");
}
```

После этого реализуем работу с метаданными ФС, описанных в заданий (`superblock`, `dentry`, `inode`):

```
void vtfs_kill_sb(struct super_block*);
struct dentry* vtfs_mount(struct file_system_type*, int, const
char*, void*);
int vtfs_fill_super(struct super_block*, void*, int);
struct inode* vtfs_get_inode(struct super_block*, const struct
inode*, umode_t, int);

struct inode* vtfs_get_inode(
    struct super_block* sb, const struct inode* dir, umode_t mode,
int i_ino
) {
  struct inode* inode = new_inode(sb);
  if (inode != NULL) {
    inode_init_owner(idmap, inode, dir, mode);
  }

  inode->i_ino = i_ino;
  return inode;
}

int vtfs_fill_super(struct super_block* sb, void* data, int silent)
{
  struct inode* inode = vtfs_get_inode(sb, NULL, S_IFDIR, 1000);
  sb->s_root = d_make_root(inode);
  if (sb->s_root == NULL) {
    return -ENOMEM;
  }
  printk(KERN_INFO "return 0\n");
  return 0;
}

void vtfs_kill_sb(struct super_block* sb) {
  printk(KERN_INFO "vtfs super block is destroyed. Unmount
successfully.\n");
}
```

И также функцию для монтировки:

```
struct dentry* vtfs_mount(
    struct file_system_type* fs_type, int flags, const char* token,
void* data
```

```
) {
  struct dentry* ret = mount_nodev(fs_type, flags, data,
vtfs_fill_super);
  if (ret == NULL) {
    printk(KERN_ERR "Can't mount file system");
  } else {
    printk(KERN_INFO "Mounted successfully");
  }
  return ret;
}
```

Что это позволяет нам делать:

- Регистрировать/снять ФС
- Получить `inode`
- Создать `superblock` при создании и регистрации ФС ядру
- Удалить `superblock` при снятии ФС
- Создать корневую директорию
- Монтировать ФС

Пока что есть основа, но работать с ФС не можем.

```
$ sudo mount -t vtfs "TODO" /mnt/vt
$ cd /mnt/vt
-bash: cd: /mnt/vt: Not a directory
```

2.3. Часть 3. Вывод файлов и директорий

На этом этапе реализовано вывод содержимого корневой директории. Это реализуется за счёт функции `lookup` и `iterate`.

```
struct file_operations vtfs_dir_ops = {
    .iterate_shared = vtfs_iterate,
};

int vtfs_iterate(struct file* flip, struct dir_context* ctx) {
  struct dentry* dentry = flip->f_path.dentry;
  struct inode* inode = dentry->d_inode;
  unsigned long offset = ctx->pos;
  ino_t ino = inode->i_ino;

  // Пока что оставим статический список файлов, потом исправлю.
Обычно это работает через
  // бесконечный цикл с верхним условием, который будет кидать
панику если что-то пойдет не так
  struct {
    const char* name;
```

```
    unsigned char ftype;
    ino_t ino;
  } entries[] = {
      {       ".", DT_DIR,                              ino},
      {      "..", DT_DIR, dentry->d_parent->d_inode->i_ino},
      // Тестовый файл, нужен будет на следующем этапе
      {"test.txt", DT_REG,                              101},
  };

  int total_entries = sizeof(entries) / sizeof(entries[0]);

  if (offset >= total_entries) {
    return 0;
  }

  for (; offset < total_entries; offset++) {
    if (!dir_emit(
            ctx,
            entries[offset].name,
            strlen(entries[offset].name),
            entries[offset].ino,
            entries[offset].ftype
        )) {
      return -ENOMEM;
    }
    ctx->pos++;
  }
  return 0;
}

struct inode_operations vtfs_inode_ops = {
    .lookup = vtfs_lookup,
};

struct dentry* vtfs_lookup(
    struct inode* parent_inode, struct dentry* child_dentry,
unsigned int flag
) {
  return NULL;
}
```

Также добавил `i_fop` и полные права для решения проблемы с выводом содержимых директории.

```
int vtfs_fill_super(struct super_block* sb, void* data, int silent)
{
  struct inode* inode = vtfs_get_inode(sb, NULL, S_IFDIR | 0777,
```

```
100);
  inode->i_op = &vtfs_inode_ops;
  inode->i_fop = &vtfs_dir_ops;
  // ...
}
```

Было реализовано всё что надо на текущем этапе, но пока что мы не можем получить список файлов.

```
$ ls /mnt/vt
ls: cannot access '/mnt/vt/test.txt': No such file or directory
test.txt
```

2.4. Часть 4. Навигация по директориям

Осуществим переход по директориям через передачу нода файла с помощью при вызове `vtfs_lookup`.

```
struct dentry* vtfs_lookup(
    struct inode* parent_inode, struct dentry* child_dentry,
unsigned int flag
) {
  ino_t root = parent_inode->i_ino;
  const char* name = child_dentry->d_name.name;
  if (root == 100 && !strcmp(name, "test.txt")) {
    struct inode* inode = vtfs_get_inode(parent_inode->i_sb, NULL,
S_IFREG, 101);
    d_add(child_dentry, inode);
    // Пока что фиговня
  } else if (root == 100 && !strcmp(name, "dir")) {
    struct inode* inode = vtfs_get_inode(parent_inode->i_sb, NULL,
S_IFDIR, 200);
    d_add(child_dentry, inode);
  }
  return NULL;
}
```

2.5. Часть 5. Создание и удаление файлов

Для реализации текущего этапа были реализованы 2 новые функций: `create` и `unlink`:

```
struct inode_operations vtfs_inode_ops = {
    .lookup = vtfs_lookup,
    .create = vtfs_create,
};

int vtfs_create(
    struct mnt_idmap* idmap,
```

```
    struct inode* parent_inode,
    struct dentry* child_dentry,
    umode_t mode,
    bool b
) {
  ino_t root = parent_inode->i_ino;
  const char* name = child_dentry->d_name.name;
  if (root == 100 && !strcmp(name, "test.txt")) {
    struct inode* inode = vtfs_get_inode(parent_inode->i_sb, NULL,
S_IFREG | 0777, 101);
    inode->i_op = &vtfs_inode_ops;
    inode->i_fop = NULL;

    d_add(child_dentry, inode);
    mask |= 1;
  } else if (root == 100 && !strcmp(name, "new_file.txt")) {
    struct inode* inode = vtfs_get_inode(parent_inode->i_sb, NULL,
S_IFREG | 07777, 102);
    inode->i_op = &vtfs_inode_ops;
    inode->i_fop = NULL;

    d_add(child_dentry, inode);
    mask |= 2;
  }
  return 0;
}

int vtfs_unlink(struct inode* parent_inode, struct dentry*
child_dentry) {
  const char* name = child_dentry->d_name.name;
  ino_t root = parent_inode->i_ino;
  if (root == 100 && !strcmp(name, "test.txt")) {
    mask &= ~1;
  } else if (root == 100 && !strcmp(name, "new_file.txt")) {
    mask &= ~2;
  }
  return 0;
}
```

В результате этого этапа мы можем:

- Создать файлы:

```
$ touch oops.txt
$ ls
oops.txt
$ touch wearecharliekirk.txt
```

```
$ ls
oops.txt wearecharliekirk.txt
```

- Проверить на существование файла:

```
$ touch wearecharliekirk.txt
$
```

- Удалить файлы:

```
$ ls
oops.txt wearecharliekirk.txt
$ rm wearecharliekirk.txt
$ ls
oops.txt
$ rm oops.txt
$ ls
```

2.6. Часть 6. Реализация с хранилищем данных в RAM

Так как мы теперь переходим в сохранение объектов в ФС в настоящую память, придется модифицировать логику их сохранения. Сначала были созданы 2 структур, соответствующих этим объектам:

```
struct vtfs_file {
  struct list_head list;
  char* name;
  ino_t ino;
  umode_t mode;
  struct inode* inode;
  size_t size;
  char* data;
};
struct vtfs_dir {
  struct list_head children;
  struct vtfs_file* self;
};
```

Потом были модифицированы функции `create`, `unlink`, `iterate`, `lookup` и `fill_super` (Полный изменения находятся в репозитории, точнее под коммитом `step 6`). Теперь функциональность нашего ФС представляет из себя минимальный набор операции, результаты которых будут сохраняться в оперативную память.

2.7. Часть 7. Создание и удаление директорий

Теперь реализуем создание и удаление директорий. Это делается через реализацию `mkdir` и `rmdir`. Опять же, они находятся в репозитории, но общий механизм их работы предельно простая:

- Сначала проверяются валидность аргументов, потом создается файл. Затем аллоциуется и копируется данные в другое место, после чего не его основе создается директория. После создания директории инициализируется метаданные в виде `inode`, режима (это важно, именно через него мы можем отличать директорию от файла), операции и другие. В самом конце присваиваем ему `inode` через `d_add`, а также увеличиваем подсчёт ссылок.
- Удалить ещё проще: после проверки сначала удаляются все объекты, принадлежащие ему, потом снимается с него `inode` через `d_drop`, а также уменьшаем подсчёт ссылок, после чего освобождается память.

2.8. Часть 8*. Чтение и запись в файлы

Здесь мы будем реализовать чтение и запись в файлов. Это делается через реализацию `read` и `write`, которые вызываются через `cat` и `echo`.
Реализация в репозитории. Как происходит чтение:

1. Сначала получается данный файла:

```
struct vtfs_file* file_data = inode->i_private;
```

Если там пусто, то возвращается 0 (EOF).

2. Вычисляется количество прочитаемых данных:

```
available = file_data->size - *ppos;
to_copy = min(len, available);
```

3. Потом копируется данные из памяти ядра в `userspace`:

```
copy_to_user(buf, file_data->data + *ppos, to_copy);
```

4. Сдвигается смещение файла (`*ppos`).

Как происходит запись:

1. Вычисляется требуемый объём:

```
new_size = max(*ppos + len, file_data->size);
```

2. Переаллоцируем файловый буфер при необходимости:

```
krealloc(file_data->data, new_size, GFP_KERNEL);
```

3. Заполняем нулями всю выделенную память, после чего его заполняем данными пользователя:

```
copy_from_user(file_data->data + *ppos, buf, len)
```

4. Сдвигается смещение файла (`*ppos`).

В результате этого этапа мы теперь можем осуществить `I/O` над файлами.

```
$ touch file1
$ echo "I wanna go home :(" > file1
$ touch file2
$ echo "ew a j*b" > file2
$ cat file1
I wanna go home :(
$ cat file2
ew a j*b
$ echo "j*b" > file1
$ cat file1
j*b
```

2.9. Часть 9*. Жёсткие ссылки

Теперь реализуем жесткие ссылки. Это тип объекта файловой системы, который указывает на содержимые оригинального файла. Отличается от копии файла тем, что указывает на одну и ту же `inode`, когда у копии есть свой `inode`.

1. Сначала получаем оригинальный файл:

```
old_file = old_dentry->d_inode->i_private;
```

2. Получаем родительскую директорию:

```
parent_dir = parent_inode->i_private;
```

3. Если оригинальный файл на самом деле является директорией, то отклоняем:

```
if (S_ISDIR(old_file->mode))
    return -EPERM;
```

4. Осуществляем проверку имен через итерацию `parent_dir->children`, чтобы убедиться, что такой имени нет.
5. Аллоцируем память на новый файл:

```
new_file = kzalloc(sizeof(struct vtfs_file), GFP_KERNEL);
```

6. Скопируем метаданные:

```
new_file->name = ...
new_file->size = old_file->size;
new_file->data = old_file->data;
new_file->inode = old_dentry->d_inode;
```

7. Добавим его в директорию и присваиваем новый `inode` (на самом деле тот же что и оригинальный), а также увеличиваем подсчёт ссылок.

2.10. Часть 10*. Сервер файловой системы

Реализация находится в директории `backend`. По сути представляет из себя обычного CRUD-приложения, который принимает запросы от нашего ФС, обрабатывает и сохраняет его, после чего возвращается ответ, который дальше обрабатывается модулем.

Сущность ФС на сервере:

```
public class FileMetadata {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String fileName;

    private long inode;

    private int linkCount;
}
```

Вызываемые эндпоинты (контроллер):

```
public class FileController {

    private final FileService fileService;

    @PostMapping("/create")
    public ResponseEntity<FileMetadata> createFile(@RequestParam
String fileName) {
        FileMetadata file = fileService.createFile(fileName);
        return new ResponseEntity<>(file, HttpStatus.CREATED);
    }

    @PostMapping("/link")
    public ResponseEntity<FileMetadata> linkFile(@RequestParam
String oldFileName, @RequestParam String newFileName) {
        FileMetadata file = fileService.linkFile(oldFileName,
newFileName);
        return new ResponseEntity<>(file, HttpStatus.CREATED);
    }

    @DeleteMapping("/delete")
    public ResponseEntity<Void> deleteFile(@RequestParam String
fileName) {
        fileService.deleteFile(fileName);
        return new ResponseEntity<>(HttpStatus.NO_CONTENT);
```

```
    }
}
```

Пример вызова API на модуле:

```
  {
    int rc = vtfs_http_create(child_dentry->d_name.name);
    if (rc != 0) {
      LOG("vtfs_create: remote create failed for '%s'\n",
child_dentry->d_name.name);
      return rc;
    }
  }
```

В итоге этого вызова получаем:

```
...
vtfs_http_create: remote created file 'charliekirk.txt', server
returned 201
...
```

Теперь это означает, что файлы хранятся на отдельном сервере, что защищает целостность наших данных от неожиданных технических проблем в самом модуле ядра.

3. Вывод

В этом лабораторном работе (которая является полностью отдельным от `xv6`) было поэтапно реализована примитивная файловая система, чтобы познакомиться с принципом её работы под капотом. Также было реализован сервер файловой системы, чтобы симулировать «облаку» при работе с файловой системой.

Каковым был весь курс? По моему честному мнению, это было одним из лучших курсов в факультете ПИиКТ с точки зрения организации. Обособление организации лабораторных занятий от лекционных и контрольных способствует этому — бывшие студенты этого курса могут перестроить содержания лабораторных работ под актуальных тем, и это очень помогает текущим обучающимся. Не без своих сложностей, очевидно, но закрыть предмет, считающимся самым сложным на третьем курсе за менее чем за месяц, и при этом вынести большое количество информации — признак хорошо организованной дисциплины. У меня есть желание стать практиком по этой дисциплине и продолжить этот цикл для студентов, так как я чувствую, что и у меня есть новые идеи и представления, которых могу передавать своим студентам.